



Python4Delphi Concepts

September 23, 2026

This will be the maXbox Starter Tutorial 197.

[maxkleiner/python4delphi](https://maxkleiner.github.io/python4delphi/): Free components that wrap up Python into Delphi and Lazarus (FPC)

Python for Delphi (P4D) is a set of free components that wrap up the Python DLL into Delphi, maXbox and Lazarus (FPC). They let you easily execute Python scripts, create new Python modules and new Python types. You can create Python extensions as DLLs and much more. P4D supports als maXbox functionality in a runtime script, which we will show it in 10 programming concepts:

```

328 function PyCode_10_ProgrammingConcepts(my_project: string): string;
329 begin
330   with TPythonEngine.Create(nil) do begin
331     try
332       loadDLL;
333       autofinalize:= false;
334       ExecString('import io,sys');
335       //execString('import comtypes.client');
336       execString('from contextlib import contextmanager');
337       //execString('from freegames import floor, square, vector');
338       execstr('output = io.StringIO()') //for print out
339       execstr('sys.stdout = output')
340     try
341       //1. Closures
342       execstr(pyClosure);

```

square: 25
 12586269025
 comprehension: [0, 4, 16, 36, 64]
 Result is 0
 Hello, Ahad
 All 10 Done:3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)]
 mX5 executed: 23/09/2026 18:02:29 Runtime: 0:0:3.402 Memload: 82% use
 RemObjects Pascal Script. Copyright (c) 2004-2026 by RemObjects Software & maXbox5

https://sourceforge.net/projects/maxbox5/files/examples/1497_10_programming_concepts.txt/download

Advertisement

PREVENTION | EDUCATION | SUPPORT | COMMUNITY | HOPE
 Your purchase helps the Marshall Kidney Foundation provide education, resources, and support for kidney patients, caregivers, and families across Broward, Miami-Dade, and Monroe counties.
Together for Healthier Kidneys. A Stronger Tomorrow.™

Snack That Gives Back

Your popcorn purchase supports kidney health education and resources for patients and caregivers.

Support the Cause

Upload Select ▼ Crop

Site title *

Remaining 11/32

[Privacy Settings](#)

This huge Delphi function embeds a Python interpreter inside a Delphi application and runs a short demo that executes 10 Python programming concepts (closures, decorators, lambdas, generators, context managers, currying via **partial**, memoization, comprehensions, exception handling, and **__call__**), captures all **print** output, and returns a status string.

[\[github\]](#)
[\[deepwiki\]](#)
[\[deepwiki\]](#)

High-level behavior

- It creates a `TPythonEngine` instance (from the `Python4Delphi` library), loads the Python DLL, and configures the interpreter.[\[github\]](#)[\[deepwiki\]](#)
- It redirects Python's `sys.stdout` to an in-memory `io.StringIO()` buffer so that all `print` output is captured instead of going to the console.[\[deepwiki\]](#)
- It then executes a series of Python code snippets (some from variables like `pyCLosure` and `pyDEco`, others built inline) that each demonstrate one of 10 concepts.[\[maxbox6.wordpress\]](#)[\[maxbox4.wordpress\]](#)
- After running the snippets, it:
 - Prints the Python version to the Delphi console (`VTAG: ...`).
 - Retrieves the captured Python output via `output.getvalue()`, converts newlines, and prints it (`out: ...`).
 - Returns a string like `"All 10 Done:<Python version>"`.
[\[deepwiki\]](#)

If any Python error occurs, it logs the exception via `writeln(ExceptionToString(...))`; if the engine itself fails, it re-raises the error.[\[deepwiki\]](#)

Advertisement



Japaner schrubben nie das WC – sie machen stattdesse..

TrendReporter24.com

[Privacy Settings](#)

What each of the 10 sections does

Inside the embedded Python interpreter, the code runs:

1. Closures

Executes `pyCLosure` (presumably defines `make_multiplier`) and then:
`double = make_multiplier(2) print(double(5)) # → 10`
 Demonstrates a function returning another function that "closes over"

a value.[\[maxbox6.wordpress\]](#)

```
1 | double = make_multiplier(2)
2 | print(double(5))    # → 10
```

1. Decorators

Executes `pyDEco` (likely defines a `logger` decorator) and then:

```
@logger def greet(name): return f"Hello, {name}"
```

`greet("Ahad")` Shows metaprogramming by wrapping a function with logging behavior.[\[maxbox6.wordpress\]](#)

2. Lambda + map numbers = [1, 2, 3, 4] squares =

```
list(map(lambda x: x**2, numbers)) print(squares)
```

 Uses an anonymous function to square each element.[\[maxbox6.wordpress\]](#)

3. Generators

```
def countdown(n): while n > 0: yield n n -= 1
```

`for num in countdown(5): print(num)` Demonstrates lazy iteration via `yield`.[\[maxbox6.wordpress\]](#)

4. Context managers (files) with open("pylog.txt", "w") as f:

```
f.write("Hello box, file!") # no explicit close
```

 Shows automatic resource management with `with`.[\[maxbox6.wordpress\]](#)

5. Currying via `functools.partial` from `functools` import `partial`

```
def power(base, exponent): return base ** exponent
square = partial(power, exponent=2) print("square:", square(5))
# → 25
```

 Pre-fills arguments to create a specialized function.

[\[maxbox6.wordpress\]](#)

6. Memoization with `lru_cache` from `functools` import `lru_cache`

```
@lru_cache def fib(n): if n < 2: return n return fib(n-1) + fib(n-2)
print(fib(50))
```

 # Fast for big inputs Caches recursive calls to speed up Fibonacci.[\[maxbox6.wordpress\]](#)

7. List comprehensions squares = [x**2 for x in range(10) if x

```
% 2 == 0] print("comprehension: ", squares)
```

 Compact, expressive list construction with filtering.[\[maxbox6.wordpress\]](#)

8. Exception handling try: result = 10 / 0 except

```
ZeroDivisionError: result = 0 finally: print(f"Result is {result}")
```

 Demonstrates controlled error handling and cleanup.

[\[maxbox6.wordpress\]](#)

9. `__call__` to make objects callable class Greeter: def

```
__call__(self, name): return f"Hello, {name}"
greet = Greeter() print(greet("Ahad"))
```

 # → Hello, Ahad Lets a class instance be used like a function.[\[maxbox6.wordpress\]](#)

```
1  1.Closure
2  def make_multiplier(factor):
3      def multiply(value):
4          return factor * value
5      return multiply
6  double = make_multiplier(2)
7  print(double(5))    # → 10
8
9
10 2.Decorator
11 def logger(func):
12     def wrapper(*args, **kwargs):
13         print(f"Calling {func.__name__}")
14         result = func(*args, **kwargs)
15         print(f"{func.__name__} returned {result!r}")
16         return result
17     return wrapper
18 @logger
19 def greet(name):
20     return f"Hello, {name}"
21 greet("Ahad")
22
23
24 3.Lambda + map
25 numbers = [1, 2, 3, 4]
26 squares = list(map(lambda x: x**2, numbers))
27 print(squares)
28
29
30 4.Generators
31 def countdown(n):
32     while n > 0:
33         yield n
34         n -= 1
35 for num in countdown(5):
36     print(num)
37
38
39 5.Context managers (files)
40 with open("pylog.txt", "w") as f:
41     f.write("Hello box, file!")    # no explicit close
42
43
44 6.Currying via functools.partial
45 from functools import partial
46 def power(base, exponent):
47     return base ** exponent
48 square = partial(power, exponent=2)
49 print("square:", square(5))    # → 25
50
51
52 7.Memoization with lru_cache
53 from functools import lru_cache
54 @lru_cache
55 def fib(n):
56     if n < 2:
```

```

57         return n
58     return fib(n-1) + fib(n-2)
59 print(fib(50)) # Fast for big inputs
60
61
62 8.List comprehensions
63 squares = [x**2 for x in range(10) if x % 2 == 0]
64 print("comprehension: ", squares)
65
66
67 9.Exception handling
68 try:
69     result = 10 / 0
70 except ZeroDivisionError:
71     result = 0
72 finally:
73     print(f"Result is {result}")
74
75
76 10.__call__ to make objects callable
77 class Greeter:
78     def __call__(self, name):
79         return f"Hello, {name}"
80 greet = Greeter()
81 print(greet("Ahad")) # → Hello, Ahad

```

I/O and return value

Advertisement



Erlebe den Landal-Herbst!

**Bauernherbst in den Bergen:
Luxus-Chalets, Almbetriebe,
Wandern & Biken. Anreisen im
Sen. & Okt. Perfekt für Familien &**

[Privacy Settings](#)

- `sys.stdout` is redirected to `output = io.StringIO()`, so all Python `print` calls go into that buffer.[\[deepwiki\]](#)
- The Delphi `maXbox` code then:
 - Prints the Python version: `print('VTag: '+evalstr('sys.version'))`
 - Retrieves and normalizes the captured output: `println('out: ' + stringreplace(evalstr('output.getvalue().strip()'), #10, #13#10, [rfReplaceAll, rfIgnoreCase]));`
 - Returns: `result := 'All 10 Done:' + EvalStr('sys.version');` So the function's return is a single-line summary including the Python version string.[\[deepwiki\]](#)

Context: Python4Delphi / maXbox style

```

1  const pyClosure =
2  'def make_multiplier(x): '+lf+
3  '    def multiply(y):      '+lf+
4  '        return x * y      '+lf+
5  '    return multiply        ';
6
7  const pyDeco =
8  'def logger(func):          '+lf+
9  '    def wrapper(*args, **kwargs): '+lf+
10 '        print(f"Calling {func.__name__}") '+lf+
11 '        return func(*args, **kwargs)      '+lf+
12 '    return wrapper                ';
13
14 function PyCode_10_ProgrammingConcepts(my_project: string)
15 begin
16     with TPythonEngine.Create(Nil) do begin
17         try
18             loadDLL;
19             autofinalize:= false;
20             ExecString('import io,sys');
21             //execString('import ctypes.client');
22             execString('from contextlib import contextmanager');
23             //execStr('from freegames import floor, square, vector
24             execstr('output = io.StringIO()') //for print out
25             execstr('sys.stdout = output')
26         try
27             //1. Closures
28             execstr(pyClosure);
29             ExecStr('double = make_multiplier(2)'+lf+
30                     'print(double(5)) # → 10 ');// # → 10')
31             //2. Decorators = Metaprogramming With Manners
32             execstr(pyDEco);
33             ExecStr('@logger'+lf+
34                     'def greet(name): '+lf+//
35                     '    return f"Hello, {name}"'+lf+
36                     'greet("Ahad")');
37
38             //3. Lambda Isn't for Show-Offs
39             ExecStr('numbers = [1, 2, 3, 4]'+lf+
40                     'squares = list(map(lambda x: x**2, numbers)'+
41                     'print(squares)');
42
43             //4. Generators Are Lazy – And That's Why They Win
44             ExecStr('def countdown(n): '+lf+
45                     '    while n > 0: '+lf+
46                     '        yield n '+lf+
47                     '        n -= 1 '+lf+
48                     '    '+lf+
49                     'for num in countdown(5):'+lf+
50                     '    print(num)');
51
52             //5. Context Managers: Automation for Setup + Clean
53             ExecStr('with open("pylog.txt", "w") as f:'+lf+
54                     '    f.write("Hello box, file!") # → no clos

```

```

55
56 //6. Currying Isn't Fancy – It's Just Pre-Filled Fur
57 execString('from functools import partial');
58 ExecStr('def power(base, exponent): '+lf+
59         '    return base ** exponent '+lf+//
60         'square = partial(power, exponent=2)'+lf+
61         'print("square:",square(5))  # → 25');
62
63 //7. Memoization: Because Redoing Work is for Amateurs
64 execString('from functools import lru_cache');
65 ExecStr('@lru_cache '+lf+
66         'def fib(n): '+lf+
67         '    if n < 2: '+lf+
68         '        return n '+lf+
69         '    return fib(n-1) + fib(n-2) '+lf+
70         ' '+lf+
71         'print(fib(50)) #Fast for big inputs');
72
73 //8. Comprehensions: The Swiss Army Knife of Python
74 execStr('squares = [x**2 for x in range(10) if x % 2');
75         'print("comprehension: ",squares)');
76
77 //9. Exception Handling Is About Controlling the Unexpected
78 ExecStr('try: '+lf+
79         '    result = 10 / 0 '+lf+
80         'except ZeroDivisionError: '+lf+
81         '    result = 0 '+lf+
82         'finally: '+lf+
83         '    print(f"Result is {result}")');
84
85 //10. The Magic of __call__: Your Class Can Act Like a Function
86 ExecStr('class Greeter: '+lf+
87         '    def __call__(self, name): '+lf+
88         '        return f"Hello, {name}" '+lf+
89         ' '+lf+
90         'greet = Greeter() '+lf+
91         'print(greet("Ahad")) # → Hello, Ahad');
92
93 print('VTAG: '+evalstr('sys.version'));//+
94 (* ExecStr('speak = comtypes.client.CreateObject("SPEAKER_05");
95         'filestream = comtypes.client.CreateObject("SPEAKER_05");
96         'filestream.open(r"'+exepath+'ccout.wav", "wb");
97         'speak.AudioOutputStream = filestream'+lf+
98         'speak.Speak("this is maxbox 5 test")'+lf+
99         'filestream.close()'); *)
100 println('out: '+stringreplace(evalstr('output.getvalue'),
101                                #10,#13#10,[rfReplaceAll, rfReplaceAll]));
102 result:= ('All 10 Done:'+(EvalStr('sys.version')));
103 except
104     writeln(ExceptionToString(ExceptionType, ExceptionInfo));
105 end;
106 except
107     raiseError;
108 finally
109     free;
110 end;
111 end;
112 end;

```

This pattern matches examples from the **Python4Delphi** ecosystem (often used in “maXbox”-style demos), where Delphi hosts Python to showcase language features or run scripts inside a Windows app.[\[github\]](#)
[\[maxbox6.wordpress\]](#)[\[maxbox4.wordpress\]](#)

Advertisement



Erlebe den Landal-Herbst!

**Bauernherbst in den Bergen:
Luxus-Chalets, Almabtriebe,
Wandern & Biken. Anreisen im
Sep. & Okt. Perfekt für Familien &**

[Privacy Settings](#)

If you share `pyCclosure` and `pyDEco`, I can explain exactly what those two snippets define and how they behave in this demo.

The exact definitions of `pyCclosure` and `pyDEco` aren’t in the snippet you posted, but in this kind of “10 Python concepts” demo they almost always follow the standard textbook examples that match the comments and subsequent calls. Based on how they’re used, here’s what they define.

`pyCclosure` – closure factory for `make_multiplier`

Based on the idea of <https://python.plainenglish.io/10-programming-concepts-that-took-me-years-to-understand-until-i-saw-these-examples>

Advertisements

Occasionally, some of your visitors may see an advertisement here, as well as a [Privacy & Cookies banner](#) at the bottom of the page.

You can hide ads completely by upgrading to one of our paid plans.

[UPGRADE NOW](#)[DISMISS MESSAGE](#)

Posted in [Code](#), [maXbox](#), [Models](#), [Tools](#)

Leave a comment